# 4 – Gestione e automazione dei simulatori di guida

Questo capitolo è dedicato al sottosistema di gestione dei simulatori di guida presenti in Esplace e, in particolare, al sistema **ACC Totem** sviluppato per semplificare la configurazione delle gare su *Assetto Corsa Competizione* [6]. L'obiettivo è fornire agli operatori uno strumento unico, accessibile da un totem o da un tablet, che permetta di impostare rapidamente sessioni di gara personalizzate, assegnare piloti e vetture ai simulatori, avviare e riavviare il server dedicato e automatizzare per quanto possibile il collegamento dei client alla sessione.

ACC Totem si inserisce così nel quadro più ampio dell'automazione del centro: mentre il Capitolo 3 ha descritto come i PC da gaming vengono riportati a uno stato standard ogni notte, qui l'attenzione si sposta sulla gestione delle esperienze di guida, che richiedono una maggiore interazione con gli utenti e una coordinazione stretta tra più postazioni. In questo contesto, i simulatori di guida rappresentano un banco di prova ideale per combinare competenze di sviluppo web, scripting e gestione di servizi con esigenze operative reali.

---

## 4.1 Contesto, infrastruttura e requisiti dei simulatori di guida

L'area sim-racing di Esplace, già descritta nel Capitolo 1, è composta da dodici postazioni di guida dedicate. Queste postazioni utilizzano hardware più performante rispetto ai normali PC gaming del centro: ogni simulatore monta un PC dedicato con GPU di fascia superiore, maggiore potenza di calcolo e memoria, collegato a un monitor curvo in formato ultrawide con risoluzione fino al 4K. Questa dotazione permette un campo visivo ampio e un'elevata qualità grafica, ma richiede al tempo stesso framerate stabili e impostazioni omogenee tra le postazioni, così da evitare differenze percepibili nell'esperienza di guida.

Le periferiche di input comprendono volanti con force feedback, pedaliere e strutture di supporto dedicate, a cui si aggiungono le palette sul volante. L'insieme di questi elementi rende i simulatori più complessi da gestire rispetto ai PC gaming tradizionali, sia per la calibrazione delle periferiche, sia per l'allineamento delle impostazioni di gioco.

I PC dei simulatori sono collegati via cavo alla stessa infrastruttura di rete utilizzata dal resto del centro. Le sessioni che coinvolgono più postazioni sono però gestite da un server dedicato, sul quale girano sia il server di *Assetto Corsa Competizione* sia il backend di ACC Totem. Quest'ultimo genera e aggiorna i file di configurazione del simulatore e controlla l'avvio e il riavvio dell'istanza di gioco. In questo modo la gestione delle gare è centralizzata, mentre le singole postazioni si limitano a collegarsi alla sessione predisposta, mantenendosi allineate alla configurazione pubblicata.

L'utilizzo dei simulatori in un centro Esports come Esplace è caratterizzato da una forte dinamicità. Nel corso di una singola giornata si alternano sessioni di prova libera, mini–gare a tempo, attività promozionali con gruppi numerosi ed eventi più strutturati, come tornei interni o iniziative legate a festival e collaborazioni esterne.

Ogni formato richiede combinazioni diverse di circuito, durata, condizioni meteo e numero di partecipanti, spesso con tempi molto ridotti tra un gruppo e il successivo.

In assenza di strumenti dedicati, la gestione di questa variabilità ricadrebbe interamente sull'operatore, che dovrebbe intervenire ogni volta direttamente sui file di configurazione del server di *Assetto Corsa Competizione*, modificando a mano parametri come tracciato, durata delle sessioni, meteo ed entry list dei piloti. Questa modalità richiede familiarità con la struttura dei file JSON di ACC, è soggetta a errori e tende a rallentare l'organizzazione degli eventi, soprattutto nei momenti di maggiore affluenza.

Da queste considerazioni derivano alcuni requisiti funzionali specifici. È necessario disporre di uno strumento che astragga la complessità dei file di configurazione dietro un'interfaccia semplice, organizzata per "eventi" anziché per singoli parametri tecnici. L'operatore deve poter definire rapidamente un gruppo di partecipanti, associare a ciascun simulatore un pilota e una vettura, scegliere il circuito e strutturare il "weekend di gara" (pratica, qualifiche, gara) senza doversi confrontare direttamente con la sintassi dei file. A supporto dell'attività in sala è utile una vista sintetica delle postazioni, che evidenzi in modo immediato quali **simulatori** sono liberi, assegnati o occupati, e un meccanismo per filtrare nomi inappropriati, soprattutto in contesti aperti a famiglie e scuole.

Un ulteriore requisito riguarda l'automazione del collegamento dei simulatori al server di gara. Se gestita manualmente, ogni sessione richiederebbe di avviare il gioco su ciascun PC, navigare nei menu fino alla sezione multiplayer, selezionare la scheda LAN, individuare la lobby corretta e inserire l'eventuale password. Ripetere questi passaggi su molte postazioni, più volte al giorno, è laborioso e sottrae tempo all'assistenza diretta ai clienti. È quindi necessario un meccanismo che, una volta definita la configurazione dell'evento, consenta ai simulatori di aggiornare automaticamente i propri file locali e di eseguire in modo semi–automatico le operazioni di join, riducendo al minimo l'intervento manuale dell'operatore e rendendo il sistema utilizzabile anche da personale non specialistico.

Infine, il sistema di gestione deve integrarsi con l'infrastruttura esistente e mantenere un comportamento robusto rispetto a aggiornamenti moderati del gioco o della configurazione hardware, così da limitare la necessità di interventi tecnici frequenti. Nei paragrafi successivi viene illustrata l'architettura del sottosistema "Simulatori" e il modo in cui ACC Totem è stato progettato per soddisfare questi vincoli.

## 4.2 Architettura del sottosistema "*ACC Totem*"

Il sistema ACC Totem è stato progettato come un sottosistema autonomo per la gestione dei simulatori, in grado di coprire l'intero ciclo di vita di una sessione di gara: dalla configurazione tramite totem fino all'avvio automatico dei client e alla raccolta dei risultati. Dal punto di vista architetturale è composto da tre elementi principali:

- **un backend Node.js/TypeScript** [8], che funge da "orchestratore" del server di *Assetto Corsa Competizione*;
- **un frontend PWA sviluppato in React** [9], utilizzato dagli operatori tramite totem o tablet;
- **un insieme di script eseguiti sui PC dei simulatori**, basati su PowerShell e AutoHotkey [13], che automatizzano il collegamento dei client (descritti più nel dettaglio nella Sezione 4.4).

Questa separazione consente di isolare la logica di interfaccia, la logica di orchestrazione lato server e l'automazione sui client, rendendo il sistema più manutenibile e semplice da estendere.

Il **frontend** è una Progressive Web App realizzata con React e Vite [9,10], pensata per l'utilizzo su totem e tablet. Si tratta di un'applicazione single-page con interfaccia ottimizzata per l'input touch, che guida l'operatore attraverso una serie di passi successivi; l'organizzazione nel dettaglio del flusso a schede è descritta nella Sezione **4.3**. In quanto PWA, l'applicazione può essere installata sul dispositivo e avviata a tutto schermo, presentandosi all'operatore come una normale applicazione dedicata. I dati statici, come il catalogo di vetture e circuiti o i preset meteo, **sono codificati** in moduli TypeScript e caricati lato client, mentre i dati variabili (configurazioni di gara, risultati, stato dei simulatori) vengono acquisiti tramite API HTTP esposte dal backend. Per alcune funzionalità, come l'aggiornamento dei risultati, sono predisposti anche stream SSE (Server-Sent Events), che potranno essere estesi alla telemetria in sviluppi futuri.

Il **backend**, scritto in TypeScript e realizzato con Node.js ed Express, un framework web minimale per applicazioni HTTP in ambiente Node.js, espone una serie di endpoint REST che coprono le funzioni principali del sistema. L'endpoint centrale è **`/session`**, utilizzato per creare o aggiornare la configurazione del server ACC a partire dai dati inseriti nel totem: nome del server, circuito, numero di giocatori, struttura delle sessioni (pratica, qualifiche, gara), parametri meteo e entry list dei piloti. Il payload in ingresso viene validato tramite schemi definiti con **Zod** [11], una libreria di validazione che permette di descrivere in modo dichiarativo la struttura attesa dei dati; in questo modo è possibile intercettare fin da subito configurazioni incoerenti o incomplete, evitando di generare file non validi per il simulatore.

Una volta validata la richiesta, il backend traduce il payload in una serie di file di configurazione ACC (ad esempio **`event.json`**, **`settings.json`**, **`entrylist.json`**). La generazione è affidata a moduli dedicati, che si occupano di mappare le scelte dell'operatore sui campi previsti dal "server handbook" di ACC [6].

La scrittura su disco avviene in formato UTF-16 LE, come richiesto dal simulatore, e viene gestita in modo atomico: i file vengono generati in una posizione temporanea e rinominati solo a scrittura completata, per ridurre il rischio di corruzioni in caso di errori o interruzioni durante l'aggiornamento.

La *Figura 6* mostra la funzione utilizzata per scrivere i file di configurazione di ACC in formato UTF-16 LE, sfruttando un file temporaneo e una rinomina finale per ottenere una scrittura atomica.

```
// fileWriters.ts (estratto)
export async function writeJsonUtf16Le(
  dir: string,
  name: string,
  obj: unknown,
) {
  await fs.mkdir(dir, { recursive: true });
  const file = path.join(dir, name);
  const json = JSON.stringify(obj, null, 2);

  const tempFile = path.join(
    dir,
    `.${name}.tmp.${crypto.randomBytes(8).toString("hex")}`,
  );

  const buf = iconv.encode(json, "utf16-le");
  await fs.writeFile(tempFile, buf, { encoding: "binary" });
  await fs.rename(tempFile, file);
}
```

*Figura 6 - Scrittura atomica dei file di configurazione ACC in UTF-16 LE (estratto da fileWriters.ts)*

Per quanto riguarda il controllo del **server ACC**, il backend utilizza **NSSM** (Non-Sucking Service Manager) [12] per gestire l'istanza del dedicated server come servizio Windows. Un modulo specifico si occupa di invocare NSSM per avviare, arrestare o riavviare il servizio; quando una nuova configurazione viene applicata, il backend effettua un riavvio controllato, assicurandosi che il server riparta con i file aggiornati. In pratica, l'operatore non deve mai interagire direttamente con la macchina che ospita ACC: tutte le operazioni passano attraverso l'interfaccia web, che a sua volta dialoga con il backend.

Oltre alla gestione delle configurazioni, il backend integra alcuni componenti di supporto. Un **results watcher** monitora la cartella dei risultati del server ACC, effettua il parsing dei file generati a fine sessione e mantiene in memoria uno storico degli ultimi eventi, esposto tramite appositi endpoint e mostrato nella scheda "Risultati" del frontend. Un insieme di API dedicate gestisce inoltre lo **stato dei rig**, permettendo al backend di registrare e comunicare agli agent installati sui simulatori le informazioni necessarie per l'auto-join, come l'indirizzo del server, la porta e l'eventuale password.

La logica di monitoraggio dei risultati (riassunta in *Figura 7*) è incapsulata in una classe dedicata (**`ResultsStore`**), che inizializza la directory da osservare, effettua una scansione iniziale e mantiene in memoria uno storico degli eventi, ordinato per data di creazione.

```
// resultsWatcher.ts (estratto)
export class ResultsStore {
  private dir: string | null = null;
  private items: ResultSummary[] = [];
  private byId = new Map<string, ResultSummary>();

  init(dir: string) {
    this.dir = dir;
    this.scanAll();
    this.watch();
  }

  list(): ResultSummary[] {
    return [...this.items].sort((a, b) => b.createdAt - a.createdAt);
  }
}
```

*Figura 7 - Struttura del results watcher lato backend (estratto da resultsWatcher.ts)*

Dal punto di vista della sicurezza, l'accesso alle funzionalità avanzate (come la modifica delle impostazioni di server, password e opzioni di gara più delicate) è protetto da una modalità "operatore", attivabile tramite inserimento di un PIN. L'autenticazione è gestita tramite un endpoint dedicato che imposta una sessione identificata da un cookie HttpOnly, in modo che il PIN non venga mai esposto al codice JavaScript lato client. Per ridurre il rischio di abusi o tentativi di brute-force è presente un meccanismo di rate limiting sui tentativi di login in un intervallo di tempo.

In ambiente di produzione interna, il backend viene esposto in HTTPS utilizzando certificati TLS autofirmati, distribuiti e installati manualmente sui dispositivi di sala; questa soluzione è adeguata al contesto di rete locale e consente comunque di garantire la cifratura del traffico. Il backend supporta inoltre una configurazione CORS a whitelist, in modo che solo le PWA autorizzate possano comunicare con le API.

Nel complesso, l'architettura di ACC Totem separa in modo netto l'interfaccia utente, la generazione e l'applicazione delle configurazioni lato server e l'automazione dei client. Questa struttura consente di evolvere ciascuna componente (ad esempio introducendo nuove opzioni di gara o diverse modalità di auto-join) senza dover modificare radicalmente le altre parti del sistema. Le possibili estensioni, come l'integrazione di dati live di telemetria nella scheda "Live", vengono discusse nelle sezioni conclusive del capitolo.

## 4.3 Workflow di configurazione e gestione delle sessioni

Dal punto di vista di chi lo utilizza, ACC Totem offre un percorso guidato che accompagna l'intera preparazione di una sessione sui simulatori, dalla definizione del gruppo di partecipanti fino alla partenza della gara e alla consultazione dei risultati. L'interfaccia è pensata per essere utilizzata principalmente dal personale di sala, ma è sufficientemente semplice da poter essere impiegata, nelle configurazioni più comuni, anche direttamente dai clienti in modalità self-service *(Figura 17)*.

Il sistema prevede due livelli di utilizzo. In modalità standard l'interfaccia è accessibile anche ai clienti: è possibile scegliere un preset di gara, impostare il nome del gruppo, selezionare il numero di partecipanti, assegnare piloti e vetture ai simulatori, scegliere il circuito e configurare, in forma semplificata, meteo e durata delle sessioni. Questa modalità è pensata per l'uso self-service, riducendo al minimo la necessità di intervento del personale. In modalità *operatore*, attivabile tramite inserimento di un PIN, vengono invece sbloccate opzioni aggiuntive: impostazione delle password del server (join, spectator, admin), controllo della visibilità della lobby, configurazione dettagliata di parametri avanzati (ad esempio limiti pista, *formation lap*, regole sui pit stop), gestione di preset personalizzati, associazione di livree custom e accesso al pannello "Stato rig" per monitorare e controllare i simulatori (pausa dell'auto-join, annullamento di un collegamento, chiusura forzata del gioco in caso di problemi). In questo modo le operazioni più delicate rimangono sotto il controllo del personale, mentre le azioni di configurazione più comuni possono essere svolte direttamente dagli utenti.

La prima fase è la configurazione del **gruppo**. Nella scheda "Gruppo" l'utente (cliente o operatore) indica il nome con cui il server verrà mostrato nella lista delle sessioni LAN di ACC, il numero di partecipanti e, se lo desidera, seleziona un preset di sessione (ad esempio "Pratica veloce", "Sprint 20'" o "Endurance 60'"). Dal punto di vista dell'utente, un preset corrisponde a un "pacchetto" di impostazioni già pronto, che precompila automaticamente i campi relativi a sessioni, meteo e regole di gara; resta comunque possibile modificare i singoli parametri prima di confermare la creazione della sessione. I preset vengono memorizzati lato client e risultano particolarmente utili per gli eventi ricorrenti, in cui la stessa combinazione di opzioni viene riutilizzata più volte nel corso della giornata o della settimana. Il nome del server è sottoposto a un filtro di termini vietati, in modo da evitare denominazioni offensive o inappropriate. Il numero di giocatori scelto in questa fase determina il numero di slot che verranno resi disponibili nelle schede successive e, di conseguenza, il numero di voci presenti nell'entry list generata dal backend.

Nella scheda **"Giocatori"** l'utente passa dall'idea astratta di "gruppo" all'assegnazione concreta dei piloti ai simulatori. Una mappa schematica delle dieci postazioni dedicate mostra lo stato di ciascun rig (libero, assegnato, occupato), permettendo di gestire anche situazioni in cui non tutti i simulatori sono disponibili. Per ogni partecipante viene scelto il simulatore su cui gareggerà, viene inserito il nome che sarà visualizzato in gara e viene selezionata la vettura da un catalogo strutturato per categoria (GT3, GT4, GT2, ecc.), corredato da immagini di anteprima. In modalità operatore è inoltre possibile associare a singoli piloti livree personalizzate, richiamando file di configurazione già presenti sul server.

Nel contesto del dedicated server di *Assetto Corsa Competizione*, l'**entry list** è il file che descrive l'elenco dei piloti autorizzati a partecipare alla sessione, specificando per ciascuno nome visualizzato, vettura utilizzata e altri metadati [6]. Piuttosto che mantenere file statici differenziati per evento, ACC Totem costruisce dinamicamente questa struttura a partire dai dati inseriti nelle schede "Gruppo" e "Giocatori": il numero di slot viene sincronizzato con il numero di partecipanti, e ogni slot contiene le informazioni necessarie (simulatore assegnato, pilota, vettura, eventuale livrea personalizzata). Il backend traduce questa struttura interna in un file JSON conforme al formato previsto da ACC, assicurando coerenza tra ciò che l'utente vede a schermo e ciò che il server utilizza effettivamente per avviare la gara, senza richiedere la manutenzione manuale di più versioni dei file di configurazione.

La scheda **"Circuiti"** consente di selezionare il tracciato su cui si svolgerà la sessione. I circuiti sono presentati in una griglia visuale con miniature, mappe del tracciato e, quando disponibile, un breve video onboard. Questo approccio permette di scegliere in modo rapido anche circuiti meno conosciuti, senza doversi ricordare i nomi o gli identificativi esatti utilizzati nei file di configurazione di ACC, semplificando l'uso sia per il personale sia per gli utenti meno esperti.

Le schede **"Difficoltà & Meteo"** e **"Sessioni"** completano la definizione del "weekend di gara". Nella prima vengono impostati il livello di aiuti alla guida (ad esempio modalità più permissive per eventi con partecipanti inesperti o configurazioni più impegnative per sessioni dedicate ad appassionati), le condizioni atmosferiche e alcuni parametri avanzati, come la presenza di pit stop obbligatori o il tipo di formazione in partenza. L'interfaccia combina preset pronti all'uso con controlli più fini su temperatura, copertura nuvolosa, intensità della pioggia e velocità di evoluzione del meteo, rendendo possibile sia la creazione di eventi "standard" sia la simulazione di condizioni particolari. Nella scheda "Sessioni" l'utente, in genere l'operatore, struttura il weekend definendo l'eventuale presenza di prove libere e qualifiche, la durata delle varie sessioni e l'ora del giorno in cui si svolgono. Alcune regole di coerenza di base, ad esempio la richiesta di almeno una sessione preparatoria prima della gara vera e propria, aiutano a evitare configurazioni incoerenti o difficilmente gestibili in sala.

Quando la configurazione è ritenuta soddisfacente, l'utente conferma i dati tramite l'azione di creazione o aggiornamento della sessione. A questo punto il frontend invia al backend un payload che riassume tutte le scelte effettuate: informazioni generali sul server, circuito, struttura del weekend, entry list dei piloti e parametri di gara. Come descritto nella Sezione 4.2, il backend valida il payload tramite Zod, traduce i dati nei file di configurazione previsti da ACC, li scrive su disco nel formato richiesto e riavvia in modo controllato il server dedicato. Dal punto di vista di chi utilizza il sistema, l'effetto è quello di un unico passaggio: dopo la conferma, il server di gara viene preparato con la configurazione appena definita, senza che sia necessario intervenire direttamente sulla macchina che lo ospita.

La fase successiva riguarda il **collegamento dei simulatori** alla sessione, che viene descritta in dettaglio nella Sezione 4.4. In sintesi, gli agent in esecuzione sui PC dei simulatori interrogano periodicamente il backend per verificare se il relativo rig è stato assegnato a una sessione attiva; in caso affermativo, recuperano le informazioni necessarie (indirizzo, porta, eventuale password), aggiornano i file locali di ACC e avviano il gioco, delegando poi ad AutoHotkey l'esecuzione dei passaggi necessari nei menu per completare il join. La scheda "Stato rig", accessibile in modalità operatore, offre una vista sintetica di questa fase: chi gestisce la sala può controllare quali postazioni stanno eseguendo l'auto-join, mettere in pausa l'automazione su singoli rig, annullare un collegamento o richiedere la chiusura del gioco in caso di anomalie.

Al termine della sessione, la scheda **"Risultati"** consente di consultare in modo rapido l'esito della gara: tempi sul giro, posizione finale, vettura utilizzata e altre informazioni essenziali vengono raccolte dal backend a partire dai file generati dal server ACC e resi disponibili in forma tabellare. Questa vista è utile sia nell'uso quotidiano dei simulatori, perché permette di recuperare in pochi secondi l'esito di una gara senza aprire direttamente i file di log, sia in contesti con molti gruppi consecutivi, in cui è necessario comunicare rapidamente podi e classifiche. A seconda delle esigenze, chi gestisce l'evento può utilizzare i risultati per presentare i vincitori al pubblico, salvare screenshot a fini promozionali o verificare il corretto completamento della sessione prima di passare al gruppo successivo.

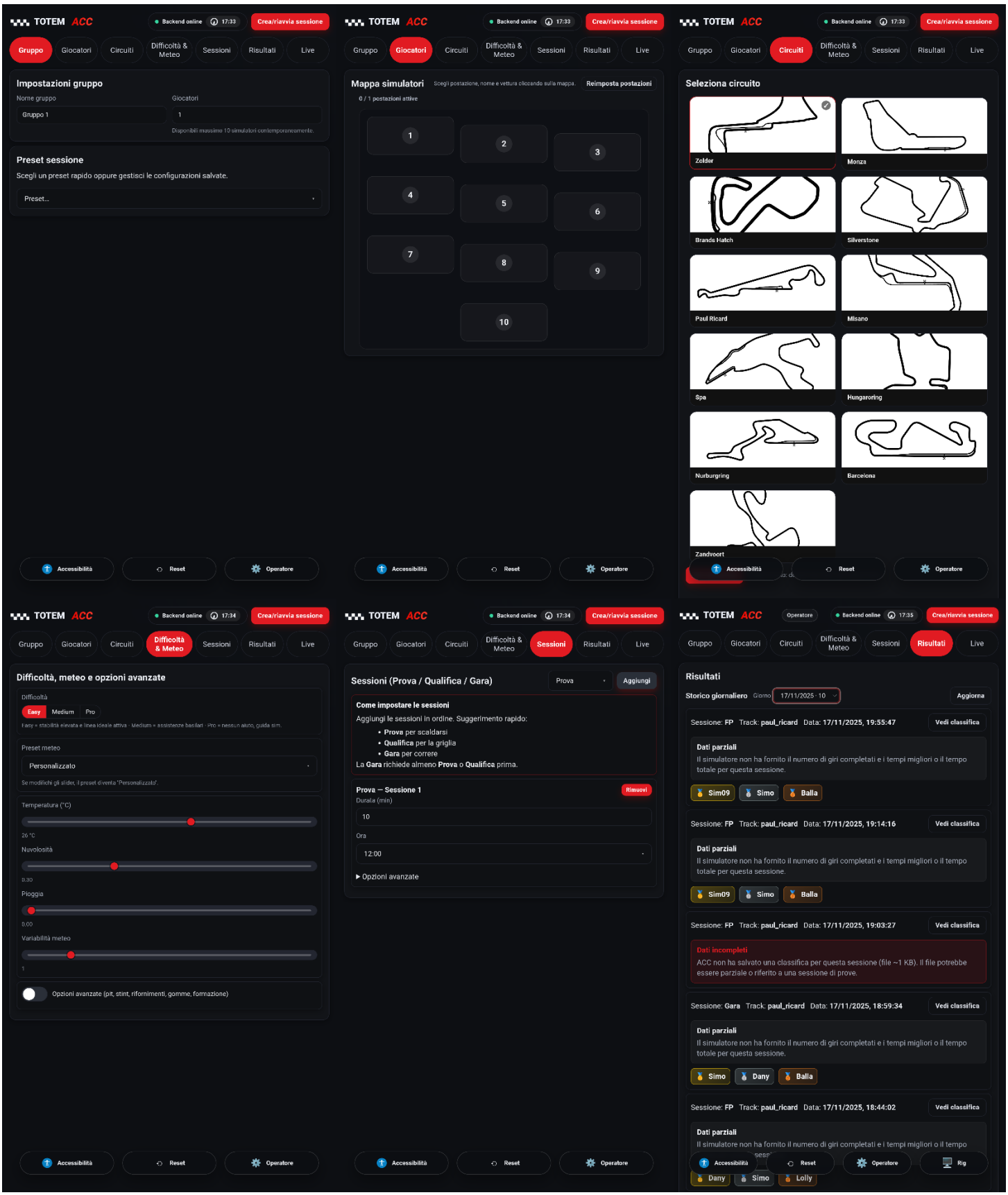


*Figura 8 – Interfaccia della PWA ACC Totem: panoramica delle schede principali di configurazione e consultazione (Gruppo, Giocatori, Circuiti, Difficoltà & Meteo, Sessioni, Risultati).*

Nel complesso, il workflow di ACC Totem riduce la gestione di una sessione a una sequenza finita di passi all'interno di un'unica interfaccia, nascondendo la complessità dei file di configurazione di ACC e delle procedure di avvio dei client. Le schede si susseguono in un ordine naturale, ma restano liberamente accessibili: è possibile, ad esempio, correggere all'ultimo momento l'assegnazione di una vettura o modificare la durata di una sessione senza dover ricominciare da capo. La scheda "Live", pensata per visualizzare dati di telemetria in tempo reale, è al momento presente in forma prototipale e non è ancora parte del flusso operativo standard; la sua integrazione con l'API di broadcast di ACC è prevista tra i possibili sviluppi futuri del sistema.

---

## 4.4 Script e procedure di automazione dei simulatori

**L'automazione** del collegamento dei simulatori al server di gara è una delle componenti più peculiari di ACC Totem. Assetto Corsa Competizione, infatti, non espone un'interfaccia a linea di comando stabile per effettuare il join a una lobby, né fornisce API ufficiali per controllare il client. Per ottenere comunque un comportamento il più possibile automatico si è adottata una soluzione ibrida, basata su chiamate HTTP verso il backend e su script che automatizzano l'interfaccia grafica del gioco.

Su ciascun PC simulatore sono presenti due script PowerShell principali: un **watcher**, eseguito in modo continuativo (**`agent-watch.ps1`**), e un **agent** vero e proprio (**`agent.ps1`**), avviato dal watcher quando viene rilevata una nuova sessione da servire. Il watcher interroga periodicamente il backend tramite API dedicate (**`/rigs/{simId}/launch-info`** e **`/rigs/{simId}/status`**), utilizzando l'identificativo del simulatore per verificare se quella postazione è stata assegnata a una sessione attiva. Se la risposta indica che il rig è assegnato e che la versione della configurazione è cambiata, il watcher avvia l'agent e, nel frattempo, mantiene aggiornato lo stato della postazione verso il server (idle, assigned, launched, paused), includendo informazioni come la presenza di processi di ACC in esecuzione e gli eventuali PID del gioco.

Il watcher gestisce anche alcuni casi particolari. Un semplice file di flag locale (ad esempio **`auto-join.pause`** nella directory degli script) permette di mettere in pausa l'auto-join su un singolo simulatore: se il file è presente, il watcher segnala lo stato di pausa al backend e si astiene dall'avviare nuove sessioni su quella macchina, finché il flag non viene rimosso. Dalla parte opposta, il backend può inviare comandi remoti, come la richiesta di chiudere ACC su un rig o di sospendere temporaneamente l'automazione per una postazione; il watcher intercetta questi segnali nelle risposte dell'API e si occupa di terminare i processi del gioco o di aggiornare lo stato in modo coerente.

Il comportamento principale del watcher può essere sintetizzato dal ciclo nella *Figura 9*, che gestisce la pausa locale, interroga il backend e avvia l'agent quando viene rilevata una nuova sessione:

```
while ($true) {
  # Pausa locale: se esiste il flag, non si avvia nulla
  if (Test-Path $pauseFlagPath) {
    Send-RigStatus 'paused' 'Avvio automatico in pausa locale'
    Start-Sleep -Seconds 2
    continue
  }

  # Lettura informazioni di lancio dal backend
  $li = Get-LaunchInfo -baseUrl $backendUrl -sid $simId

  if ($li -and $li.ok -and $li.assigned) {
    $ver = [int64]$li.version
    if ($ver -gt $lastVersion) {
      Write-Info "Nuova sessione rilevata (version=$ver > $lastVersion)"
      Start-AgentOnce -cfgPath $resolvedConfigPath
      $lastVersion = $ver
      Start-Sleep -Seconds 10  # evita doppi avvii ravvicinati
    }
  }

  Start-Sleep -Seconds 2
}
```

*Figura 8 – Ciclo principale dello script watcher*

Quando il watcher rileva una nuova configurazione da servire, delega il lavoro all'agent. Lo script **`agent.ps1`** ha il compito di preparare il client per il join alla sessione. In primo luogo legge il file di configurazione locale (`config.json`) e, se previsto, completa alcune informazioni mancanti interrogando nuovamente il backend (porta TCP effettiva, password di join e, se disponibile, il modello di vettura assegnato al rig). Successivamente aggiorna i file di configurazione del client di ACC, in particolare `serverList.json` nella cartella dei documenti dell'utente, in modo che il server di gara appaia nella lista delle sessioni LAN del gioco. Prima di procedere all'avvio, l'agent verifica la presenza di eventuali istanze residue di ACC e, se necessario, ne forza la chiusura per garantire una partenza "pulita".

Il listato in *Figura 10* mostra la parte dello script che aggiorna `serverList.json` e avvia ACC tramite Steam con i parametri ricevuti dal backend.

```
# agent.ps1 (estratto)
$docs      = [Environment]::GetFolderPath('MyDocuments')
$accCfgDir = Join-Path $docs 'Assetto Corsa Competizione\Config'
if (-not (Test-Path $accCfgDir)) {
  New-Item -ItemType Directory -Force -Path $accCfgDir | Out-Null
}

$serverList = @{ servers = @(@{ leagueServerIP = $serverIp }) } |
  ConvertTo-Json -Depth 4
$serverListPath = Join-Path $accCfgDir 'serverList.json'
$serverList | Out-File -FilePath $serverListPath -Encoding UTF8

$steamArgs = @('-applaunch','805550')
if ($tcpPort -and $password -ne $null) {
  $steamArgs += ("-connect={0}:{1}" -f $serverIp, $tcpPort)
  if ($password -ne '') {
    $steamArgs += ("-password=$password")
  }
}

Start-Process -FilePath $steamExe -ArgumentList $steamArgs | Out-Null
```

*Figura 9 – Estratto di agent.ps1 e che aggiorna serverList.json e avvia ACC tramite Steam*

Dopo l'avvio del client, lo stesso script attende che ACC sia effettivamente in esecuzione e, se sul sistema è disponibile **AutoHotkey** [13], avvia una macro dedicata (**`acc-join.ahk`**) che completa automaticamente l'ingresso nella lobby LAN. La macro è stata tarata sulle impostazioni grafiche dei simulatori (risoluzione, scaling, layout dei menu) e riproduce la sequenza di azioni che un operatore compirebbe manualmente: raggiunge il menu multiplayer, seleziona la scheda LAN, individua il server configurato, conferma la scelta dell'auto e, se necessario, inserisce la password. Alcuni parametri, come il modello di vettura o la password stessa, vengono passati alla macro dall'agent tramite argomenti a riga di comando, così da adattare il comportamento allo scenario specifico senza dover registrare una sequenza diversa per ogni combinazione.

Dal lato frontend, la modalità operatore include un pannello "Stato rig" che offre una vista sintetica dell'intero processo di auto-join. Questa interfaccia interroga periodicamente le API di stato esposte dal backend e mostra, per ciascun simulatore, se è stato assegnato a una sessione, se ACC risulta in esecuzione, se l'auto-join è in pausa e, se presente, un eventuale messaggio descrittivo. Dallo stesso pannello l'operatore può inviare richieste di pausa o di annullamento verso singole postazioni, oppure chiedere la chiusura del gioco in caso di problemi, demandando al watcher l'esecuzione effettiva di queste azioni. In questo modo gli script di automazione rimangono confinati sui PC dei simulatori, mentre il controllo operativo avviene da un'unica interfaccia centralizzata, coerente con il modello di gestione remota adottato per il resto dell'infrastruttura.

## 4.5 Esperienza d'uso, criticità e miglioramenti apportati

L'adozione di ACC Totem ha modificato in modo significativo il modo in cui gli operatori gestiscono le sessioni sui simulatori. Prima, la preparazione di una gara richiedeva la modifica manuale dei file di configurazione del dedicated server, l'avvio uno per uno dei client e una buona familiarità con l'interfaccia di Assetto Corsa Competizione. Con ACC Totem questi passaggi sono stati condensati in pochi step sul totem: scelta del preset o della configurazione di base, associazione dei piloti alle postazioni, selezione del circuito e conferma. Questo riduce in modo sensibile il tempo necessario per allestire anche un singolo gruppo di partecipanti e, allo stesso tempo, rende più semplice gestire turni ravvicinati senza dover riconfigurare manualmente ogni postazione.

Dal punto di vista degli utenti finali, l'esperienza risulta più coerente e professionale: i partecipanti si trovano inseriti nella sessione corretta, con nome e vettura già impostati, e non devono interagire con i menu del gioco. Questo semplifica anche l'utilizzo ordinario dei simulatori in sala e riduce gli errori dovuti a configurazioni improvvisate; allo stesso tempo si rivela particolarmente utile nei contesti con un alto turnover di persone, come festival, giornate promozionali o attività con scuole, in cui la rapidità con cui si riesce a far partire una nuova gara è un elemento chiave e non è realistico chiedere agli utenti di configurare autonomamente il simulatore.

Nel corso dello sviluppo e dell'utilizzo sono emerse però alcune criticità. L'affidabilità della macro AutoHotkey dipende dalla stabilità dell'interfaccia di ACC: aggiornamenti del gioco, variazioni nelle impostazioni di risoluzione o di scaling del desktop possono richiedere una ricalibrazione delle coordinate e dei tempi della sequenza automatizzata. Per limitare questi problemi è importante mantenere le postazioni dei simulatori il più possibile uniformi dal punto di vista hardware e delle impostazioni grafiche, e prevedere una procedura di verifica ogni volta che il gioco viene aggiornato. Analogamente, anche il sistema di auto-join deve convivere con elementi non completamente controllabili, come la necessità che Steam sia attivo con un utente loggato o che il sistema non venga interrotto da aggiornamenti del sistema operativo: in questi casi, il pannello "Stato rig" e la possibilità di intervenire manualmente restano un supporto essenziale.

Sul versante backend, alcuni componenti richiedono particolare attenzione man mano che il sistema evolve. Il watcher dei risultati si appoggia alla struttura dei file JSON generati dal server ACC e alla posizione della cartella dei log: cambiamenti introdotti da nuove versioni del simulatore possono rendere necessario un adattamento del parser o della logica di scansione. Allo stesso modo, le API che espongono lo stato dei rig e i servizi di polling degli agent devono essere monitorati nel tempo per evitare che modifiche alla configurazione di rete o dei certificati HTTPS compromettano la comunicazione con i simulatori.

Nonostante questi limiti, ACC Totem ha raggiunto l'obiettivo principale: rendere scalabile la gestione dei simulatori di guida in un ambiente reale, riducendo la complessità operativa e offrendo un'esperienza più consistente agli utenti. Tra le possibili evoluzioni rientrano una maggiore integrazione con le API ufficiali del simulatore, in particolare con l'API di broadcast, al fine di completare la scheda "Live" con dati telemetrici in tempo reale, e un collegamento più stretto fra i dati raccolti (risultati, tempi, utilizzo dei rig) e gli altri sottosistemi del centro, come la raccolta dei feedback o i log di rete. In questa prospettiva, ACC Totem rappresenta una base già operativa su cui poter costruire ulteriori funzionalità, mantenendo al centro l'idea di una gestione automatizzata e centralizzata dei simulatori.